Міністерство освіти і науки України
Національний університет «Полтавська політехніка імені Юрія Кондратюка»
Навчально-науковий інститут інформаційних технологій та робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

ЗВІТ

з лабораторної роботи №3
з дисципліни «Технології на платформі .NET»

на тему: «Entity Framework Core, реляційна та нереляційна бази даних, шаблон проєктування «Репозиторій»»

Виконав: студент групи 402-ТН
Желудков Віталій Віталійович

Перевірив: Тютюнник П. Б.

Полтава — 2026

1. Опис виконаної роботи

Роботу виконано в гілці `lab3`, створеній від гілки `lab2`. Для зберігання даних використано Entity Framework Core 8 і реляційну СУБД SQLite: робота розробляється і здається на різних комп'ютерах, тому файл бази даних `Database/gamestudio.db` додано до git-репозиторію. Рішення `GameStudio.sln` містить п'ять проєктів:

Проєкт	Призначення
`GameStudio.Common`	доменна модель (Employee, Programmer, Artist, GameProject, StorePage, BugReport), інтерфейси `IRepository<T>`, `ICrudServiceAsync<T>`, CRUD-сервіси
`GameStudio.Infrastructure`	**новий:** моделі БД (папка Models), `GameStudioContext`, конфігурації Fluent API, міграції, репозиторій, перетворювачі доменна модель ↔ модель БД
`GameStudio.Nosql`	**новий (додаткове завдання):** репозиторій для MongoDB
`GameStudio.Console`	консольний застосунок: за замовчуванням демонстрація ЛР3; аргументи `lab1`, `lab2`, `nosql`, `--reset`
`GameStudio.Tests`	модульні тести NUnit (ЛР2 + нові тести CRUD-сервісу з БД)

1.1. Модель бази даних (папка Infrastructure/Models)

Для кожного класу доменної моделі створено відповідний клас моделі БД з властивостями. Первинний ключ `Id` — ціле число, яке генерує БД. Додатково є альтернативний ключ `PublicId` (унікальний), що дорівнює Guid-ідентифікатору доменного об'єкта.

Доменний клас	Модель БД	Таблиця
`Employee` (абстрактний)	`EmployeeModel` (абстрактний)	Employees
`Programmer : Employee`	`ProgrammerModel : EmployeeModel`	Programmers (TPT)
`Artist : Employee`	`ArtistModel : EmployeeModel`	Artists (TPT)
`GameProject`	`GameProjectModel`	GameProjects
`StorePage` (новий, ЛР3)	`StorePageModel`	StorePages
`BugReport`	`BugReportModel`	BugReports
— (список `TeamMemberIds`)	`ProjectAssignmentModel` (проміжна сутність)	ProjectAssignments

Зв'язки моделі (описані через Fluent API у класах папки `Configurations`):

Тип зв'язку	Таблиці	Зовнішній ключ / реалізація	При видаленні
один-до-одного	GameProjects ↔ StorePages	`StorePages.GameProjectId` + унікальний індекс	CASCADE
один-до-багатьох	GameProjects → BugReports	`BugReports.GameProjectId` (обов'язковий)	CASCADE
один-до-багатьох	Programmers → BugReports	`BugReports.AssigneeId` (необов'язковий)	SET NULL

Тип зв'язку	Таблиці	Зовнішній ключ / реалізація	При видаленні
багато-до-багатьох	GameProjects ↔ Employees	таблиця `ProjectAssignments` (складений PK) з полями Role, AssignedAt	CASCADE
наслідування TPT	Employees ← Programmers, Artists	`Programmers.Id`, `Artists.Id` = PK і FK на `Employees.Id`	CASCADE

Також налаштовано: обмеження довжини рядків (`HasMaxLength`), обов'язкові поля (`IsRequired`), збереження enum як тексту (`HasConversion<string>()`), грошові поля як REAL (SQLite не має типу decimal), індекси (`HasIndex`), CHECK-обмеження `WishlistCount >= 0`, автоматичне завантаження навігаційних властивостей (`AutoInclude`).

1.2. Міграція, репозиторій і CRUD-сервіс

- **Міграція** `InitialCreate` створена командою `dotnet ef migrations add` і застосована до SQLite командою `dotnet ef database update` (розділ 2). Під час розробки контекст створює `GameStudioContextFactory` (`IDesignTimeDbContextFactory`), а інструмент `dotnet-ef` додано як локальний (`.config/dotnet-tools.json`, відновлюється командою `dotnet tool restore`). Консольний застосунок під час запуску також викликає `Database.MigrateAsync()`.
- **Репозиторій** — інтерфейс `IRepository<T>` з завдання (у Common) і узагальнена реалізація `Repository<T>` на основі `GameStudioContext` (в Infrastructure). Розширення `IEntityRepository<T>` додає пошук за `PublicId`, пагінацію на рівні БД (`LIMIT/OFFSET`) і `SaveChangesAsync`.
- **CRUD-сервіс** — `CrudServiceAsync<TDomain, TModel>` реалізує оновлений інтерфейс `ICrudServiceAsync<T>` і працює з даними тільки через репозиторій. Перетворення між доменною моделлю і моделлю БД виконують `EmployeeMapper`, `GameProjectMapper`, `BugReportMapper`. Create / Update / Remove реєструють зміни, а `SaveAsync` зберігає їх однією транзакцією. DbContext не потокобезпечний, тому сервіс впорядковує звернення семафором `SemaphoreSlim`, і його можна викликати з кількох потоків.
- **Консольний застосунок** застосовує міграції, заповнює порожню БД згенерованими даними (`Parallel` + `CreateNew` → CRUD-сервіс), читає дані з БД новими контекстами, демонструє оновлення, видалення з каскадом, пагінацію, запити `Include` / `ThenInclude` за всіма типами зв'язків, SQL для TPT і агрегацію на стороні БД (розділ 5).

1.3. Додаткове завдання: нереляційна БД MongoDB

Проєкт `GameStudio.Nosql` містить `MongoRepository<T>` — реалізацію того самого `IRepository<T>` для MongoDB (через офіційний `MongoDB.Driver`, без EF). Дані з SQLite переносяться у 2 колекції: `employees` (Programmer і Artist в одній колекції, тип — у полі-дискримінаторі `_t`) та `projects`, де сторінка в магазині, баги і команда вкладені в документ проєкту. Для `Id = 0` новий `_id` видає атомарний лічильник у колекції `counters`. Для

перевірки використано MongoDB 8.0 (розділ 6). Рядок підключення задається змінною середовища `GAMESTUDIO_MONGO` (за замовчуванням `mongodb://localhost:27017`).

1.4. Модульні тести

Додано 10 тестів CRUD-сервісу, який працює через репозиторій з EF Core: кожен тест створює окремий тимчасовий файл SQLite і застосовує справжню міграцію. Разом із тестами ЛР2 — 49 тестів, усі пройшли (`dotnet test`, розділ 7).

Гілка lab3:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab/tree/lab3

Репозиторій:
https://github.com/vitalikcraft666-design/NUPP_NET_2026_402_TK_Zheludkov_Lab

2. Створення та застосування міграції

Міграцію створено командою `dotnet ef migrations add InitialCreate --project GameStudio.Infrastructure` (файли `Migrations/20261021213932_InitialCreate.cs`, `.Designer.cs` і `GameStudioContextModelSnapshot.cs`). Застосування до порожньої БД SQLite:

```
$ dotnet ef migrations list --project GameStudio.Infrastructure
Build started...
Build succeeded.
20261021213932_InitialCreate (Pending)

$ dotnet ef database update --project GameStudio.Infrastructure
Build started...
Build succeeded.
Applying migration '20261021213932_InitialCreate'.
Done.
```

3. ERD-діаграма бази даних

Діаграму побудовано за фактичною схемою створеної БД (метадані SQLite) у нотації «воронячої лапки» (Crow's Foot): ‖ — рівно один, ○| — нуль або один, ○< — нуль або багато. PK — первинний ключ, FK — зовнішній ключ, AK — альтернативний ключ, UQ — унікальний індекс. Пунктирні стрілки TPT — наслідування «таблиця на тип».

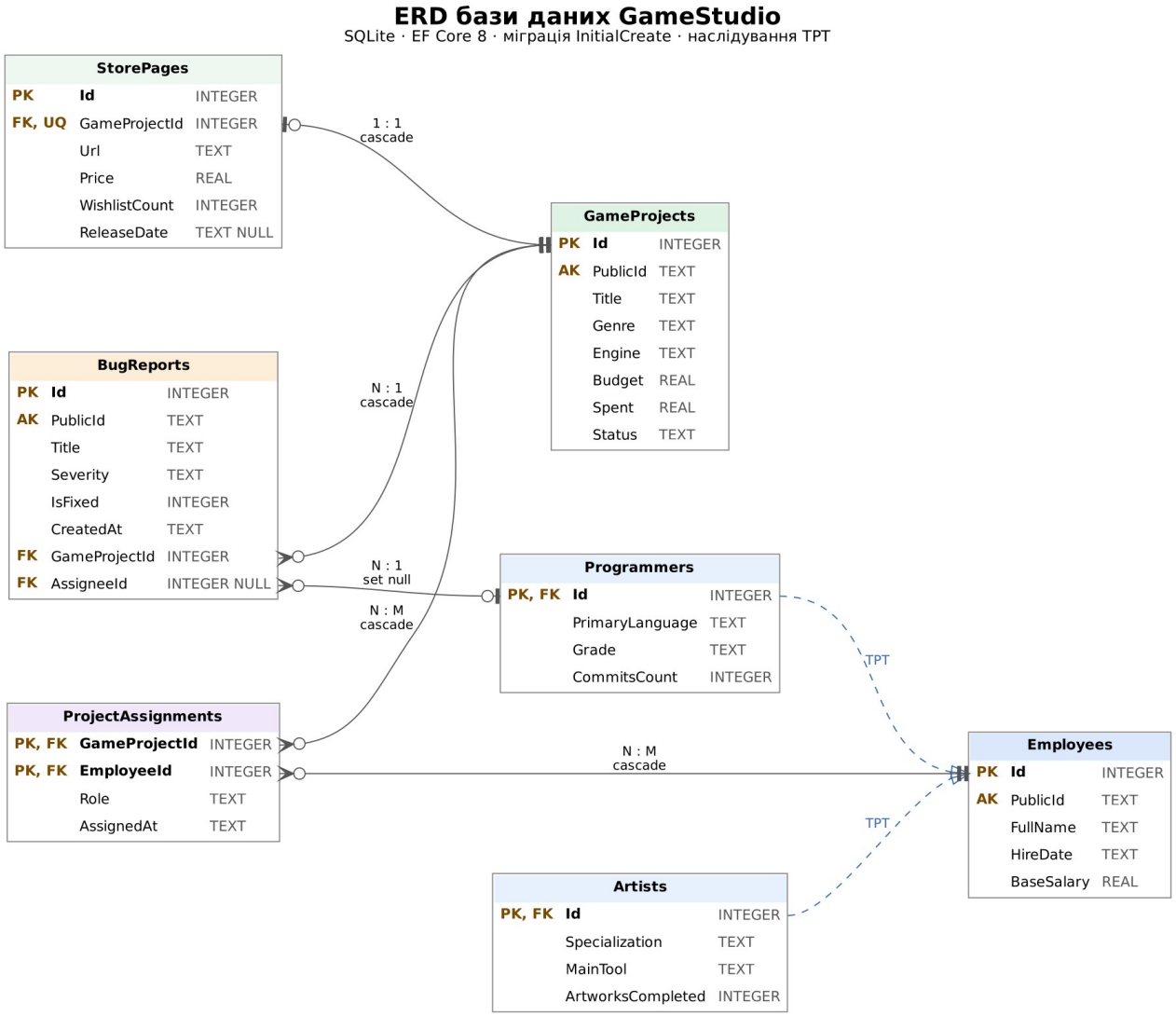


Рисунок 1 — ERD-діаграма бази даних GameStudio

4. Скріншоти створеної бази даних

Файл `Database/gamestudio.db` відкрито в DB Browser for SQLite. Структура БД: таблиці, стовпці, індекси.

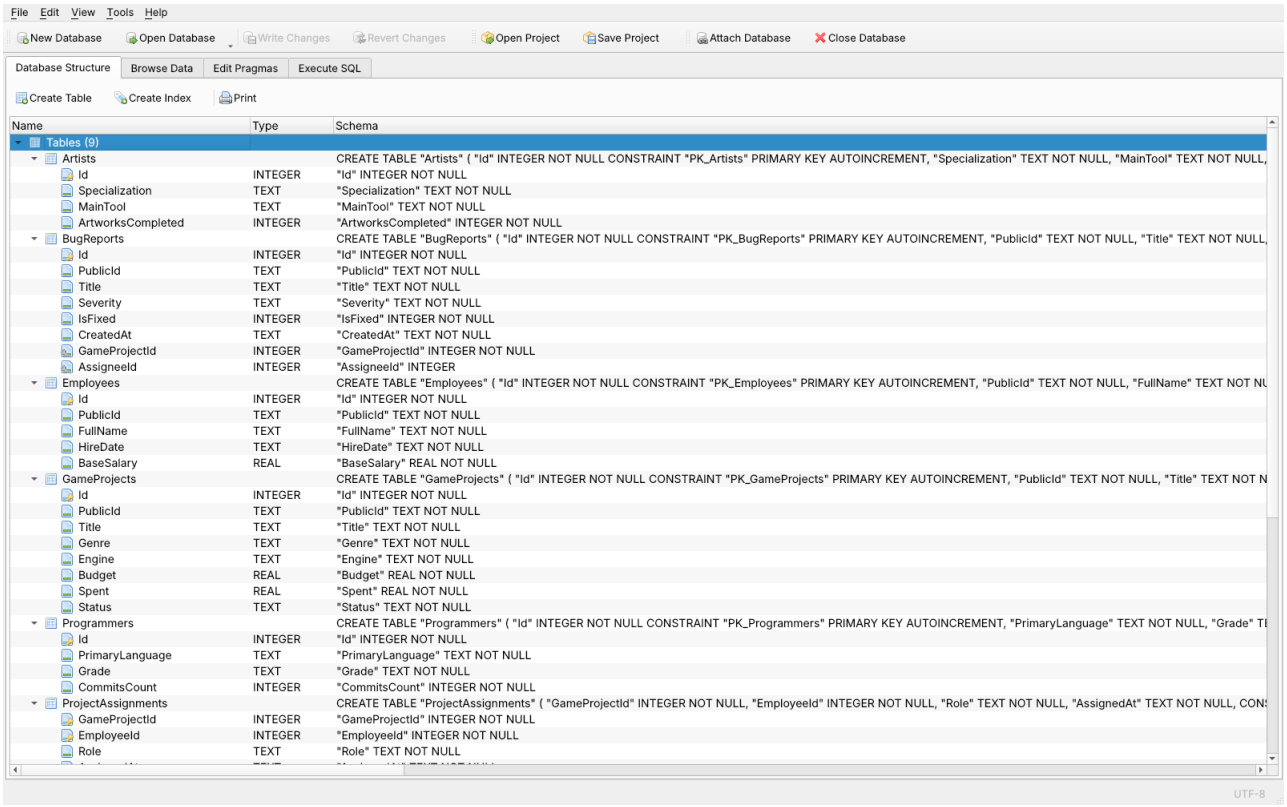


Рисунок 2 — Структура БД (Database Structure), частина 1

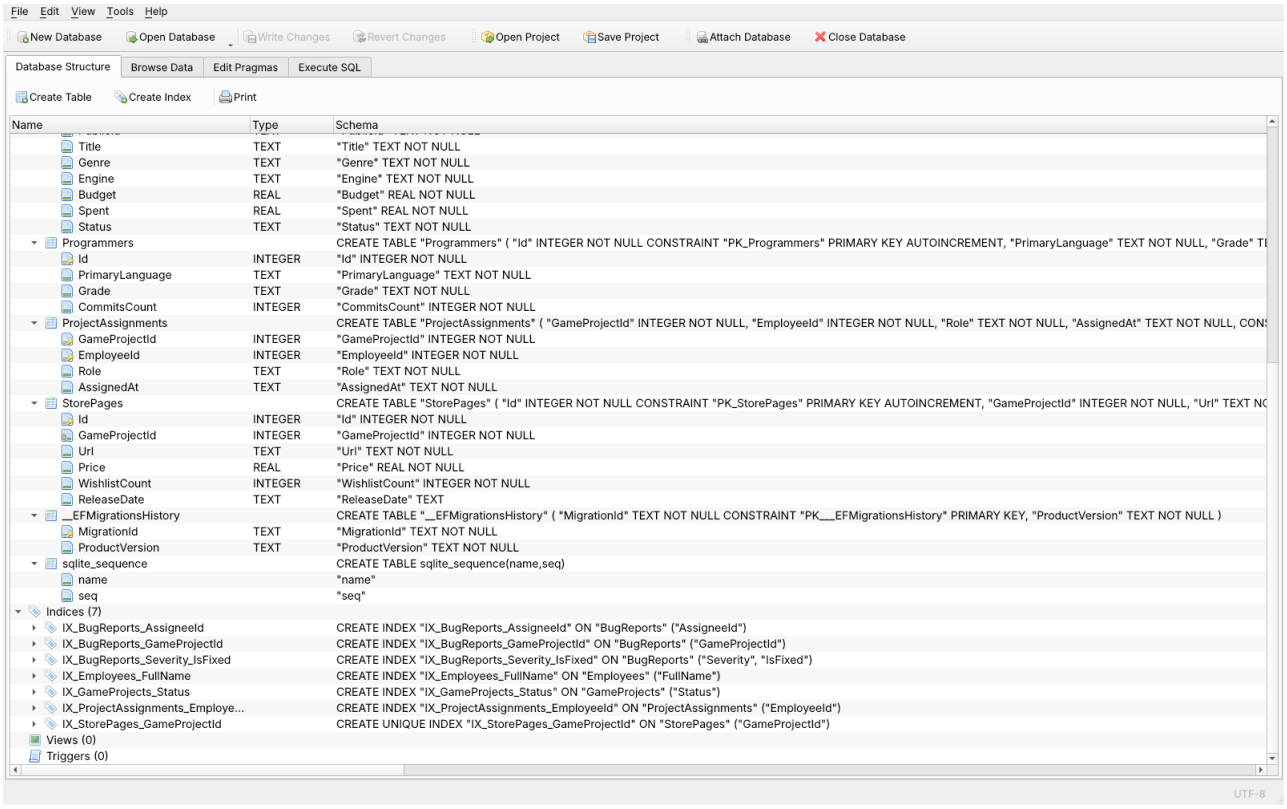
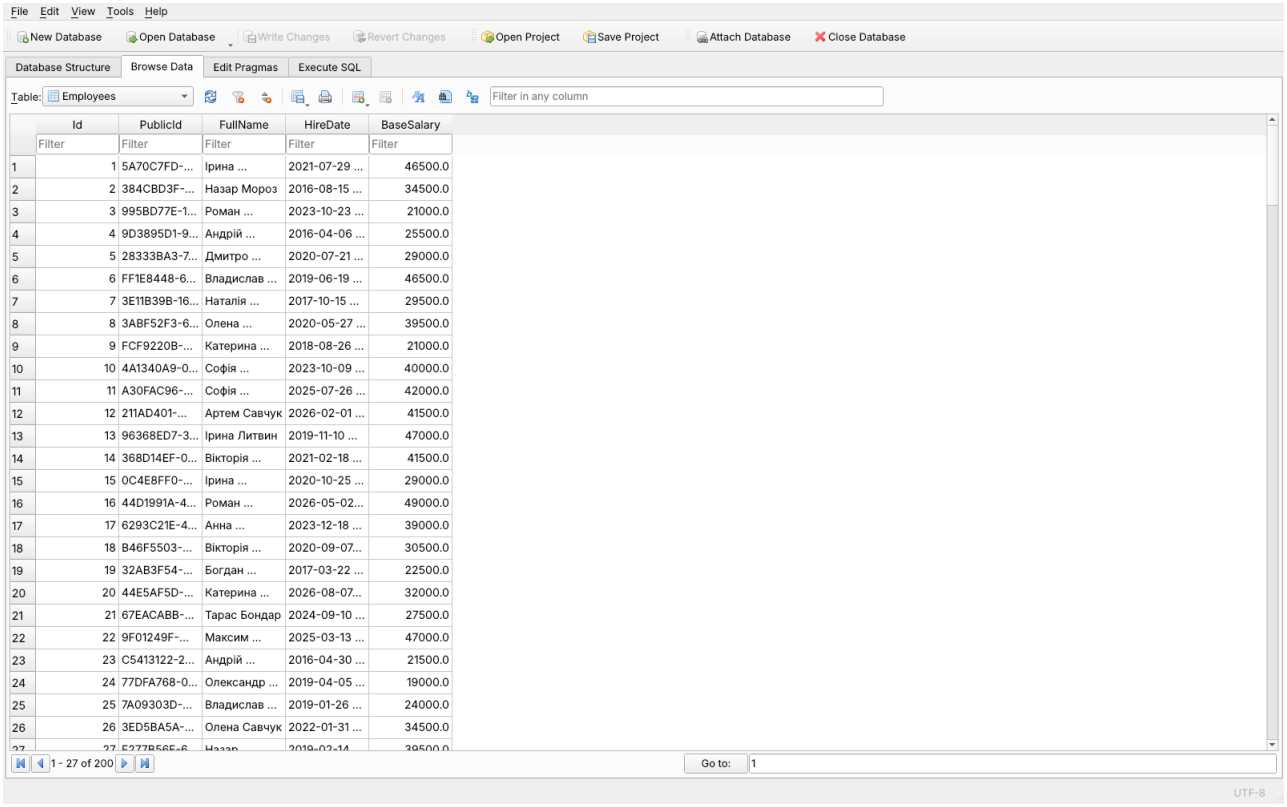


Рисунок 3 — Структура БД (Database Structure), частина 2: таблиці та індекси



File Edit View Tools Help

New Database | Open Database | Write Changes | Revert Changes | Open Project | Save Project | Attach Database | Close Database

Database Structure | Browse Data | Edit Pragmas | Execute SQL

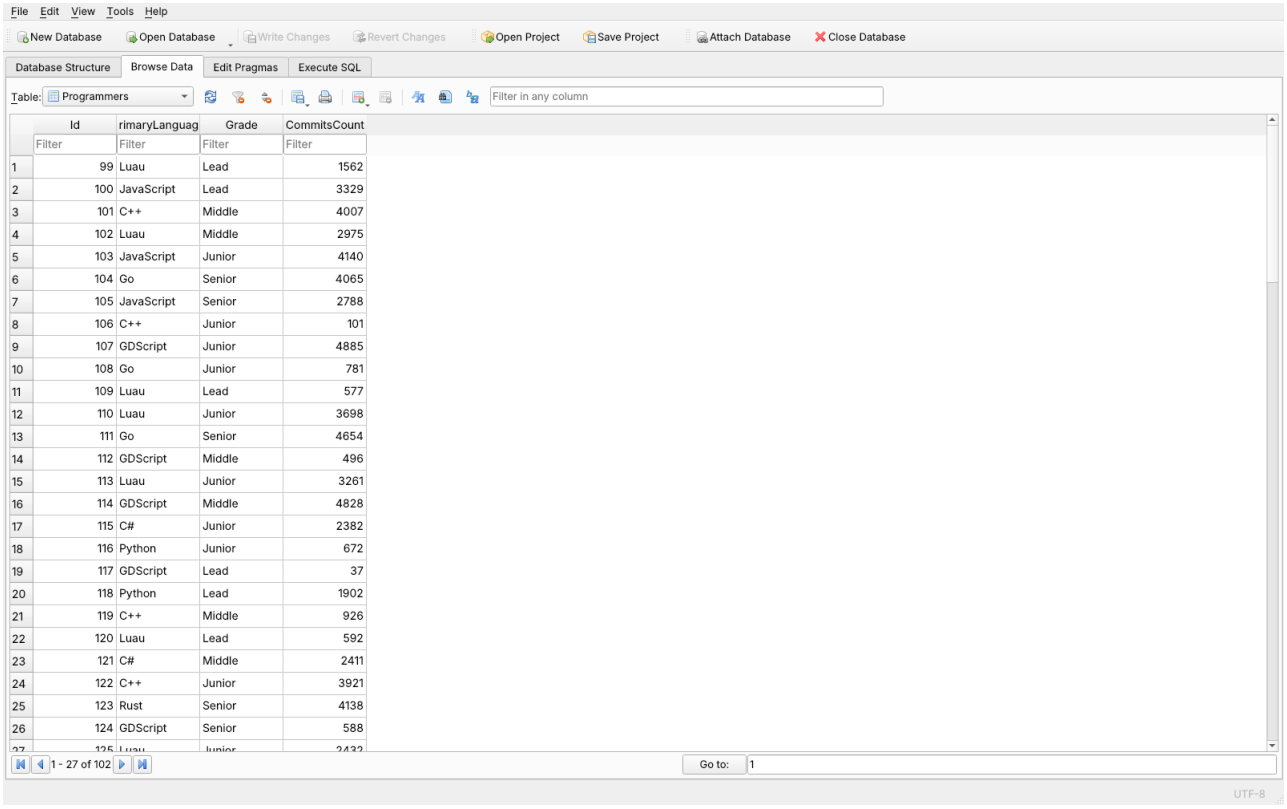
Table: Employees — Filter in any column

	Id	PublicId	FullName	HireDate	BaseSalary
	Filter	Filter	Filter	Filter	Filter
1	1	5A70C7FD-...	Ірина ...	2021-07-29 ...	46500.0
2	2	384CBD3F-...	Назар Мороз	2016-08-15 ...	34500.0
3	3	995BD77E-1...	Роман ...	2023-10-23 ...	21000.0
4	4	9D3895D1-9...	Андрій ...	2016-04-06 ...	25500.0
5	5	28333BA3-7...	Дмитро ...	2020-07-21 ...	29000.0
6	6	FF1E8448-6...	Владислав ...	2019-06-19 ...	46500.0
7	7	3E11B39B-16...	Наталія ...	2017-10-15 ...	29500.0
8	8	3ABF52F3-6...	Олена ...	2020-05-27 ...	39500.0
9	9	FCF9220B-...	Катерина ...	2018-08-26 ...	21000.0
10	10	4A1340A9-0...	Софія ...	2023-10-09 ...	40000.0
11	11	A30FAC96-...	Софія ...	2025-07-26 ...	42000.0
12	12	211AD401-...	Артем Савчук	2026-02-01 ...	41500.0
13	13	96368ED7-3...	Ірина Литвин	2019-11-10 ...	47000.0
14	14	368D14EF-0...	Вікторія ...	2021-02-18 ...	41500.0
15	15	0C4E8FF0-...	Ірина ...	2020-10-25 ...	29000.0
16	16	44D1991A-4...	Роман ...	2026-05-02...	49000.0
17	17	6293C21E-4...	Анна ...	2023-12-18 ...	39000.0
18	18	B46F5503-...	Вікторія ...	2020-09-07...	30500.0
19	19	32AB3F54-...	Богдан ...	2017-03-22 ...	22500.0
20	20	44E5AF5D-...	Катерина ...	2026-08-07...	32000.0
21	21	67EACABB-...	Тарас Бондар	2024-09-10 ...	27500.0
22	22	9F01249F-...	Максим ...	2025-03-13 ...	47000.0
23	23	C5413122-2...	Андрій ...	2016-04-30 ...	21500.0
24	24	77DFA768-0...	Олександр ...	2019-04-05 ...	19000.0
25	25	7A09303D-...	Владислав ...	2019-01-26 ...	24000.0
26	26	3ED5BA5A-...	Олена Савчук	2022-01-31 ...	34500.0

1 - 27 of 200 — Go to: 1

UTF-8

Рисунок 4 — Таблиця Employees — спільні поля всіх працівників (TPT)



File Edit View Tools Help

New Database | Open Database | Write Changes | Revert Changes | Open Project | Save Project | Attach Database | Close Database

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: Programmers — Filter in any column

	Id	rimaryLanguag	Grade	CommitsCount
	Filter	Filter	Filter	Filter
1	99	Luau	Lead	1562
2	100	JavaScript	Lead	3329
3	101	C++	Middle	4007
4	102	Luau	Middle	2975
5	103	JavaScript	Junior	4140
6	104	Go	Senior	4065
7	105	JavaScript	Senior	2788
8	106	C++	Junior	101
9	107	GDScript	Junior	4885
10	108	Go	Junior	781
11	109	Luau	Lead	577
12	110	Luau	Junior	3698
13	111	Go	Senior	4654
14	112	GDScript	Middle	496
15	113	Luau	Junior	3261
16	114	GDScript	Middle	4828
17	115	C#	Junior	2382
18	116	Python	Junior	672
19	117	GDScript	Lead	37
20	118	Python	Lead	1902
21	119	C++	Middle	926
22	120	Luau	Lead	592
23	121	C#	Middle	2411
24	122	C++	Junior	3921
25	123	Rust	Senior	4138
26	124	GDScript	Senior	588

1 - 27 of 102 — Go to: 1

UTF-8

Рисунок 5 — Таблиця Programmers — поля програмістів (Id = Employees.Id)

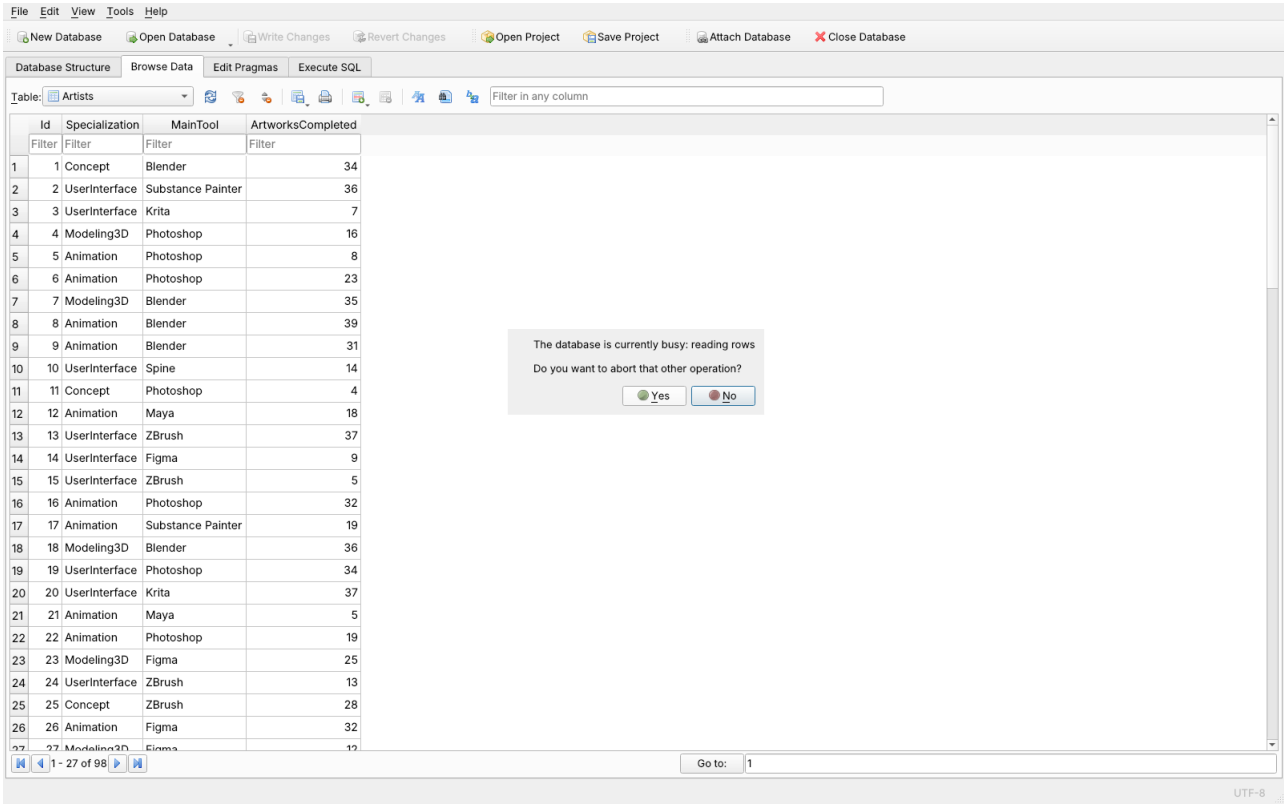


Рисунок 6 — Таблиця Artists — поля художників (Id = Employees.Id)

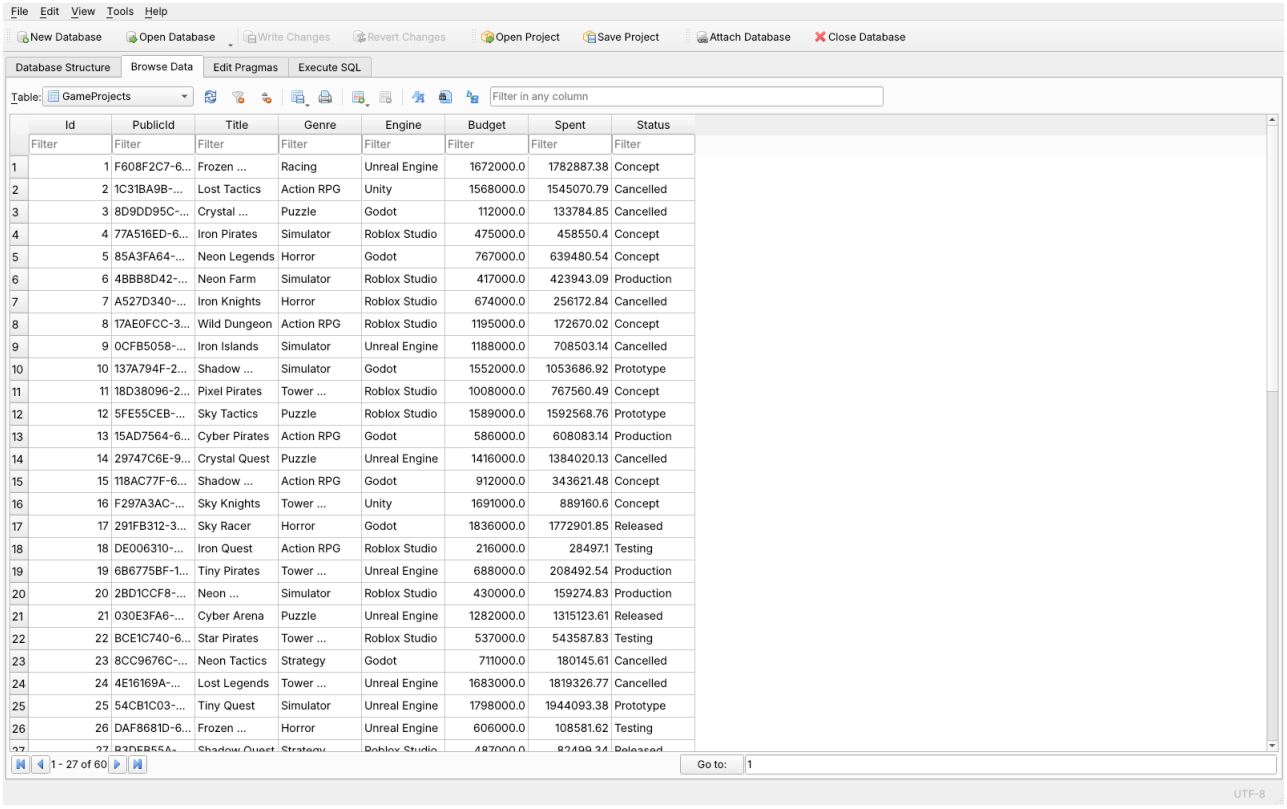
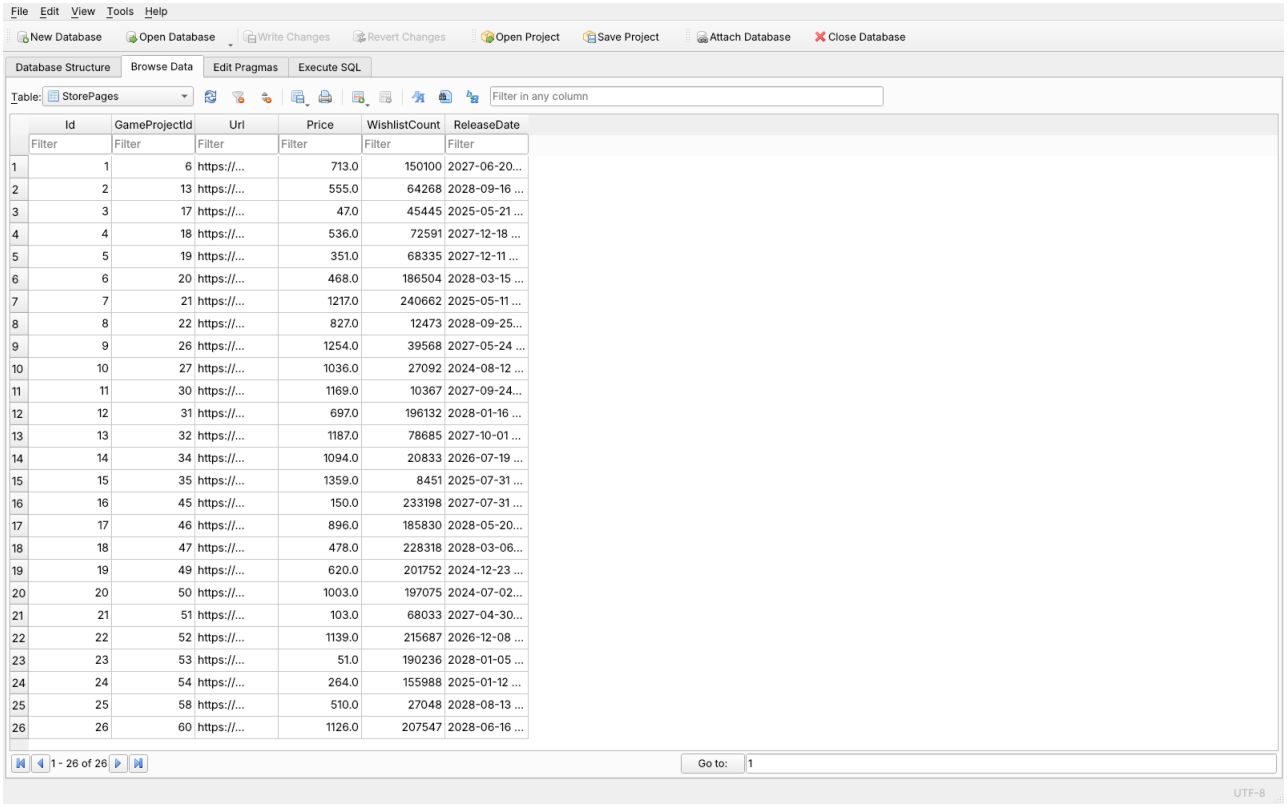


Рисунок 7 — Таблиця GameProjects



File Edit View Tools Help

New Database | Open Database | Write Changes | Revert Changes | Open Project | Save Project | Attach Database | Close Database

Database Structure | Browse Data | Edit Pragmas | Execute SQL

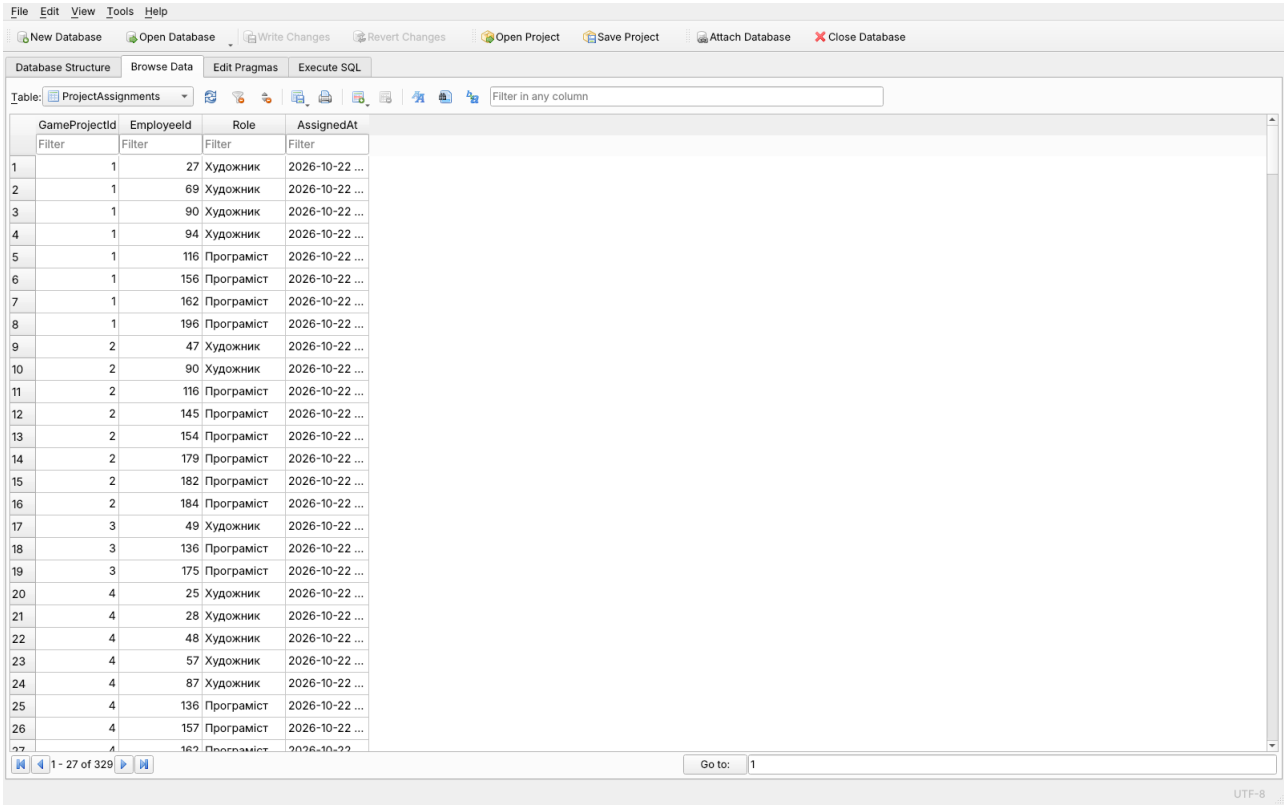
Table: StorePages — Filter in any column

	Id	GameProjectId	Url	Price	WishlistCount	ReleaseDate
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	6	https://...	713.0	150100	2027-06-20...
2	2	13	https://...	555.0	64268	2028-09-16 ...
3	3	17	https://...	47.0	45445	2025-05-21 ...
4	4	18	https://...	536.0	72591	2027-12-18 ...
5	5	19	https://...	351.0	68335	2027-12-11 ...
6	6	20	https://...	468.0	186504	2028-03-15 ...
7	7	21	https://...	1217.0	240662	2025-05-11 ...
8	8	22	https://...	827.0	12473	2028-09-25...
9	9	26	https://...	1254.0	39568	2027-05-24 ...
10	10	27	https://...	1036.0	27092	2024-08-12 ...
11	11	30	https://...	1169.0	10367	2027-09-24...
12	12	31	https://...	697.0	196132	2028-01-16 ...
13	13	32	https://...	1187.0	78685	2027-10-01 ...
14	14	34	https://...	1094.0	20833	2026-07-19 ...
15	15	35	https://...	1359.0	8451	2025-07-31 ...
16	16	45	https://...	150.0	233198	2027-07-31 ...
17	17	46	https://...	896.0	185830	2028-05-20...
18	18	47	https://...	478.0	228318	2028-03-06...
19	19	49	https://...	620.0	201752	2024-12-23 ...
20	20	50	https://...	1003.0	197075	2024-07-02...
21	21	51	https://...	103.0	68033	2027-04-30...
22	22	52	https://...	1139.0	215687	2026-12-08 ...
23	23	53	https://...	51.0	190236	2028-01-05 ...
24	24	54	https://...	264.0	155988	2025-01-12 ...
25	25	58	https://...	510.0	27048	2028-08-13 ...
26	26	60	https://...	1126.0	207547	2028-06-16 ...

1 - 26 of 26 — Go to: 1

UTF-8

Рисунок 8 — Таблиця StorePages — зв'язок один-до-одного з GameProjects



File Edit View Tools Help

New Database | Open Database | Write Changes | Revert Changes | Open Project | Save Project | Attach Database | Close Database

Database Structure | Browse Data | Edit Pragmas | Execute SQL

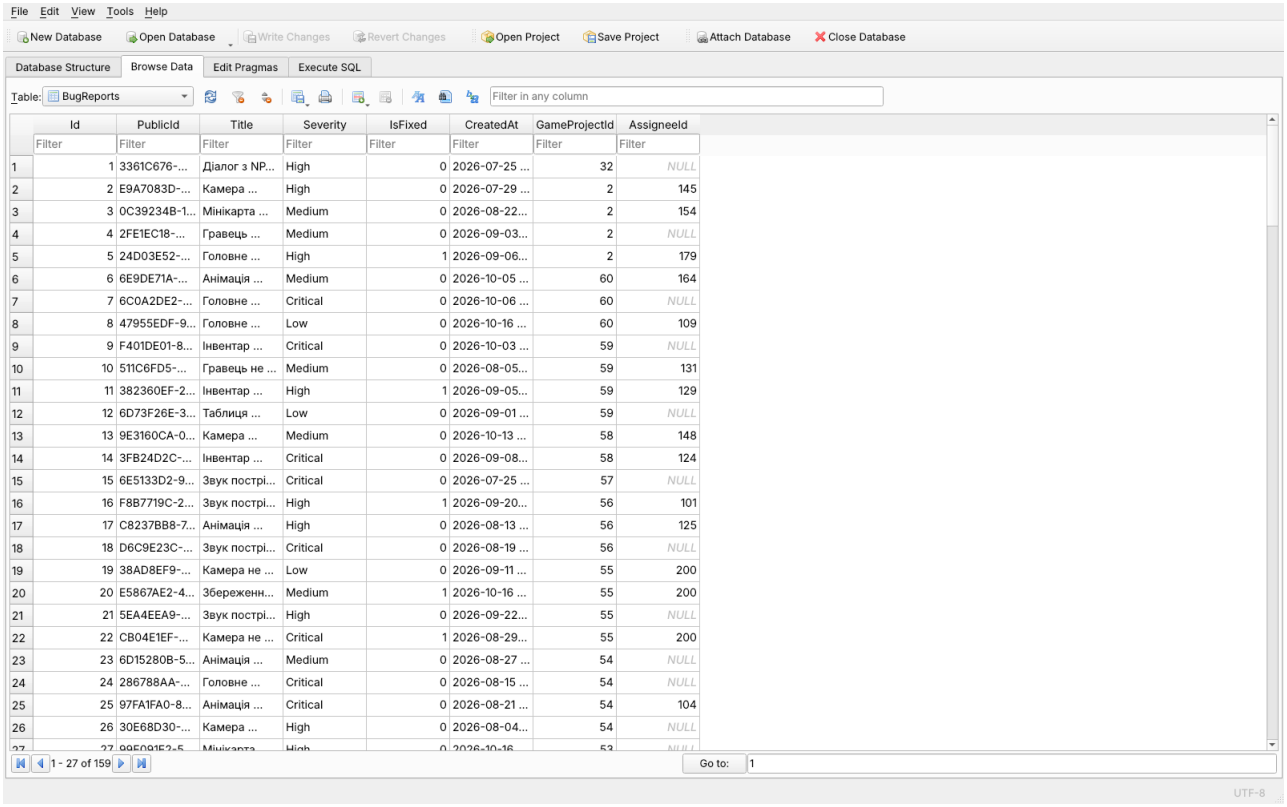
Table: ProjectAssignments — Filter in any column

	GameProjectId	EmployeeId	Role	AssignedAt
	Filter	Filter	Filter	Filter
1	1	27	Художник	2026-10-22 ...
2	1	69	Художник	2026-10-22 ...
3	1	90	Художник	2026-10-22 ...
4	1	94	Художник	2026-10-22 ...
5	1	116	Програміст	2026-10-22 ...
6	1	156	Програміст	2026-10-22 ...
7	1	162	Програміст	2026-10-22 ...
8	1	196	Програміст	2026-10-22 ...
9	2	47	Художник	2026-10-22 ...
10	2	90	Художник	2026-10-22 ...
11	2	116	Програміст	2026-10-22 ...
12	2	145	Програміст	2026-10-22 ...
13	2	154	Програміст	2026-10-22 ...
14	2	179	Програміст	2026-10-22 ...
15	2	182	Програміст	2026-10-22 ...
16	2	184	Програміст	2026-10-22 ...
17	3	49	Художник	2026-10-22 ...
18	3	136	Програміст	2026-10-22 ...
19	3	175	Програміст	2026-10-22 ...
20	4	25	Художник	2026-10-22 ...
21	4	28	Художник	2026-10-22 ...
22	4	48	Художник	2026-10-22 ...
23	4	57	Художник	2026-10-22 ...
24	4	87	Художник	2026-10-22 ...
25	4	136	Програміст	2026-10-22 ...
26	4	157	Програміст	2026-10-22 ...
27	4	162	Програміст	2026-10-22 ...

1 - 27 of 329 — Go to: 1

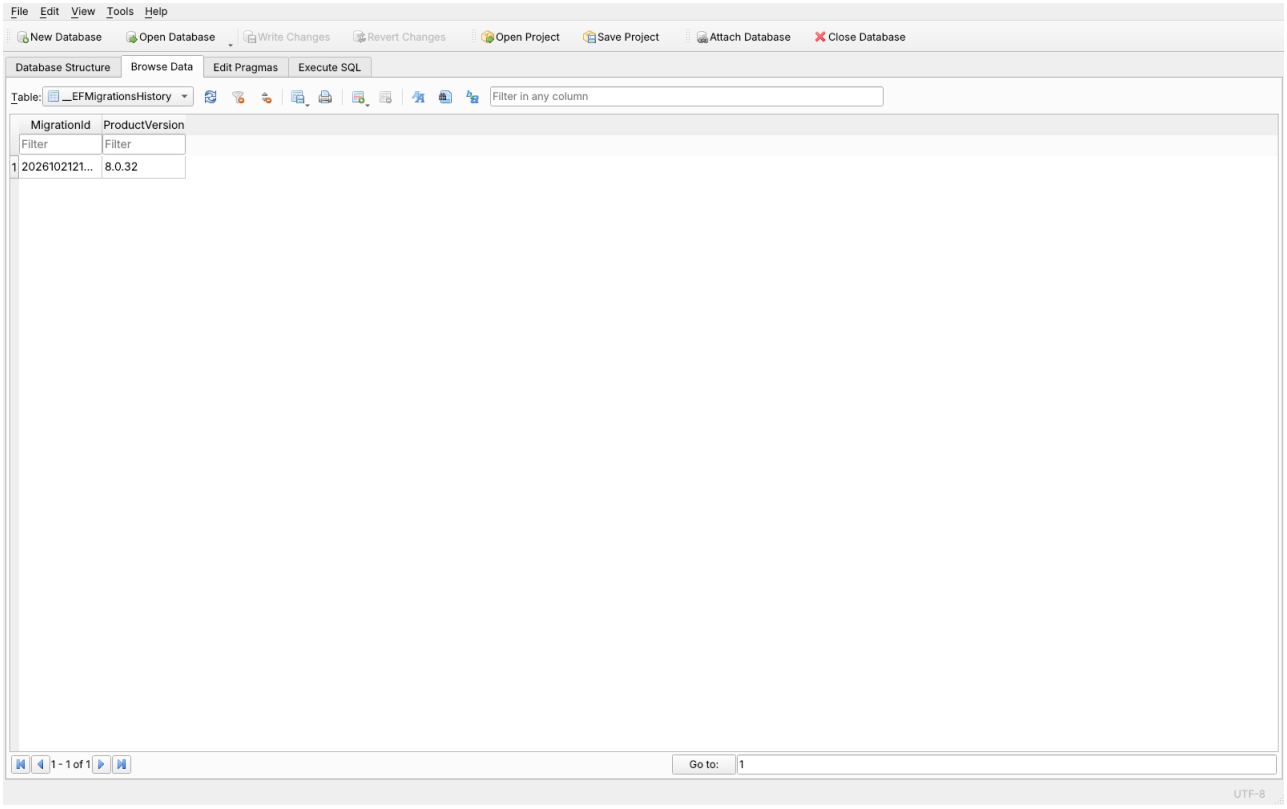
UTF-8

Рисунок 9 — Таблиця ProjectAssignments — зв'язок багато-до-багатьох



	Id	PublicId	Title	Severity	IsFixed	CreatedAt	GameProjectId	AssigneeId
1	1	3361C676-...	Діалог з NP...	High	0	2026-07-25 ...	32	NULL
2	2	E9A7083D-...	Камера ...	High	0	2026-07-29 ...	2	145
3	3	0C39234B-1...	Мінікарта ...	Medium	0	2026-08-22...	2	154
4	4	2FE1EC18-...	Гравець ...	Medium	0	2026-09-03...	2	NULL
5	5	24D03E52-...	Головне ...	High	1	2026-09-06...	2	179
6	6	6E9DE71A-...	Анімація ...	Medium	0	2026-10-05 ...	60	164
7	7	6C0A2DE2-...	Головне ...	Critical	0	2026-10-06 ...	60	NULL
8	8	47955EDF-9...	Головне ...	Low	0	2026-10-16 ...	60	109
9	9	F401DE01-8...	Інвентар ...	Critical	0	2026-10-03 ...	59	NULL
10	10	511C6FD5-...	Гравець не ...	Medium	0	2026-08-05...	59	131
11	11	382360EF-2...	Інвентар ...	High	1	2026-09-05...	59	129
12	12	6D73F26E-3...	Таблиця ...	Low	0	2026-09-01 ...	59	NULL
13	13	9E3160CA-0...	Камера ...	Medium	0	2026-10-13 ...	58	148
14	14	3FB24D2C-...	Інвентар ...	Critical	0	2026-09-08...	58	124
15	15	6E5133D2-9...	Звук постріл...	Critical	0	2026-07-25 ...	57	NULL
16	16	F8B7719C-2...	Звук постріл...	High	1	2026-09-20...	56	101
17	17	C8237BB8-7...	Анімація ...	High	0	2026-08-13 ...	56	125
18	18	D6C9E23C-...	Звук постріл...	Critical	0	2026-08-19 ...	56	NULL
19	19	38AD8EF9-...	Камера не ...	Low	0	2026-09-11 ...	55	200
20	20	E5867AE2-4...	Збереженн...	Medium	1	2026-10-16 ...	55	200
21	21	5EA4EEA9-...	Звук постріл...	High	0	2026-09-22...	55	NULL
22	22	CB04E1EF-...	Камера не ...	Critical	1	2026-08-29...	55	200
23	23	6D15280B-5...	Анімація ...	Medium	0	2026-08-27 ...	54	NULL
24	24	286788AA-...	Головне ...	Critical	0	2026-08-15 ...	54	NULL
25	25	97FA1FA0-8...	Анімація ...	Critical	0	2026-08-21 ...	54	104
26	26	30E68D30-...	Камера ...	High	0	2026-08-04...	54	NULL

Рисунок 10 — Таблиця BugReports — зв'язки один-до-багатьох



	MigrationId	ProductVersion
1	2026102121...	8.0.32

Рисунок 11 — Таблиця __EFMigrationsHistory — застосовані міграції

SQL-запит, що об'єднує таблиці за всіма зв'язками (GameProjects, ProjectAssignments, Employees, Programmers, Artists, StorePages, BugReports):

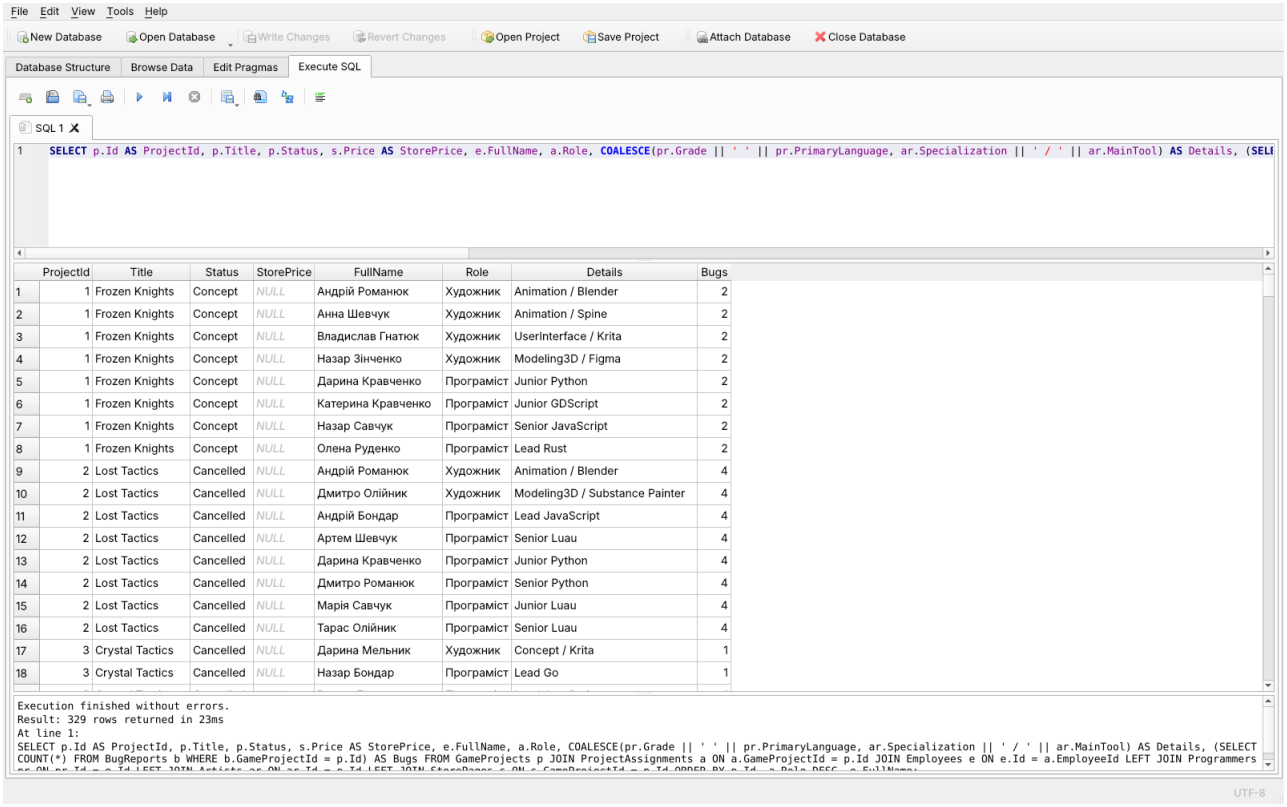


Рисунок 12 — Результат запиту з JOIN (329 рядків)

5. Результати виконання консольного застосунку

Команда запуску: `dotnet run --project GameStudio.Console` (БД щойно створена міграцією, тому застосунок заповнює її даними).

```
Лабораторна робота №3 — Entity Framework Core, SQLite, шаблон «Репозиторій» (GameStudio)
Запуск: 22.10.2026 00:46

================================================================================================
1. База даних SQLite і міграції EF Core
================================================================================================
Міграції, що очікують застосування: немає
Файл БД: Database/gamestudio.db
Застосовані міграції: 20261021213932_InitialCreate
Таблиці: Artists, BugReports, Employees, GameProjects, Programmers, ProjectAssignments, StorePages, __EFMigrationsHistory

================================================================================================
2. Заповнення БД: Parallel + CreateNew → CrudServiceAsync → репозиторій → SQLite
================================================================================================
Працівники: додано 200 з 200, SaveAsync → True (програмістів: 102, художників: 98)
Проєкти: додано 60 з 60, SaveAsync → True (зі сторінкою в магазині: 26, призначень у команди: 329)
Баги: додано 159 з 159, SaveAsync → True (призначено виконавця: 105)
Час заповнення: 2067 мс

================================================================================================
3. Читання даних із БД через CrudServiceAsync<TDomain, TModel> (нові DbContext)
================================================================================================
ReadAllAsync: працівників — 200 (Programmer: 102, Artist: 98), проєктів — 60, багів — 159
Пагінація працівників по 5 (LIMIT/OFFSET у SQL): сторінок — 40
Сторінка 1:
  1. Концепт-художник             Ірина Лисенко          стаж: 5 р., зарплата: 61 975,00 грн
  2. UI-художник                  Назар Мороз            стаж: 10 р., зарплата: 53 850,00 грн
  3. UI-художник                  Роман Коваленко        стаж: 2 р., зарплата: 24 010,00 грн
  4. 3D-моделер                   Андрій Коваленко       стаж: 10 р., зарплата: 37 150,00 грн
  5. Аніматор                     Дмитро Мороз           стаж: 6 р., зарплата: 36 220,00 грн
Сторінка 2:
  1. Аніматор                     Владислав Бондар       стаж: 7 р., зарплата: 62 015,00 грн
  2. 3D-моделер                   Наталія Павленко       стаж: 9 р., зарплата: 46 215,00 грн
  3. Аніматор                     Олена Кравченко        стаж: 6 р., зарплата: 56 360,00 грн
  4. Аніматор                     Катерина Романюк       стаж: 8 р., зарплата: 33 790,00 грн
  5. UI-художник                  Софія Кравченко        стаж: 3 р., зарплата: 47 100,00 грн
Проєкти, сторінка 1 по 3:
  1. "Frozen Knights" (Racing, Unreal Engine) — Concept, бюджет: 1 672 000 грн, витрачено: 1 782 887 грн, у команді: 8
  2. "Lost Tactics" (Action RPG, Unity) — Cancelled, бюджет: 1 568 000 грн, витрачено: 1 545 071 грн, у команді: 8
  3. "Crystal Tactics" (Puzzle, Godot) — Cancelled, бюджет: 112 000 грн, витрачено: 133 785 грн, у команді: 3
ReadAsync(4bbb8d42-008f-4361-b44f-8b7c0adbfdcb):
  "Neon Farm" (Simulator, Roblox Studio) — Production, бюджет: 417 000 грн, витрачено: 423 943 грн, у команді: 5
  сторінка в магазині: https://store.gamestudio.ua/games/neon-farm-355 — 713 грн, у бажаному: 150 100, вихід 20.06.2027
  команда: 5 осіб; багів у проєкті: 4

================================================================================================
4. CreateAsync / UpdateAsync / RemoveAsync + SaveAsync (зміни потрапляють у БД лише після SaveAsync)
================================================================================================
UpdateAsync → True, SaveAsync → True: Дарина Мороз: Senior Luau-програміст → Lead Luau-програміст (прочитано новим
контекстом)
CreateAsync(новий проєкт «Demo Arena») → True; SaveAsync → True
CreateAsync(той самий Id) → False (дублікат)
CreateAsync(баг для нового проєкту) → True; SaveAsync → True
Рядків для проєкту в БД до видалення: 4
```

```
RemoveAsync → True; SaveAsync → True
Рядків для проєкту в БД після видалення: 0 (StorePages, ProjectAssignments і BugReports видалено каскадно — ON DELETE CASCADE)
ReadAsync(видалений Id)  → KeyNotFoundException: Елемент з Id 8c0c78e7-95ce-46ea-82a6-ac8e5ff9519d не знайдено.
Баг з неіснуючим проєктом → InvalidOperationException: Проєкт з Id 61954feb-08e5-457e-83a6-3d5faf8c1352 не знайдено в БД.
ReadAllAsync(0, 5)       → ArgumentOutOfRangeException: Номер сторінки має бути не менше 1. (Parameter 'page')

================================================================================================
5. Зв'язки між таблицями: жадібне завантаження Include / ThenInclude
================================================================================================

--- Один-до-одного: GameProjects ↔ StorePages (проєкт із найбільшим списком бажаного)
    «Cyber Arena» (Released) → https://store.gamestudio.ua/games/cyber-arena-455
    ціна: 1 217 грн, у бажаному: 240 662, вихід: 11.05.2025

--- Один-до-багатьох: GameProjects → BugReports (проєкт із найбільшою кількістю багів)
    «Lost Tactics» — багів: 4
      [High    ] Камера скидається після виходу — відкрито, виконавець: Андрій Бондар
      [Medium  ] Мінікарта мерехтить у темних локаціях — відкрито, виконавець: Тарас Олійник
      [Medium  ] Гравець мерехтить у темних локаціях — відкрито, виконавець: не призначено
      [High    ] Головне меню провалюється крізь текстури — виправлено, виконавець: Марія Савчук

--- Багато-до-багатьох: GameProjects ↔ Employees через ProjectAssignments (команда того ж проєкту)
      Андрій Бондар            Програміст  (ProgrammerModel), з 22.10.2026
      Артем Шевчук             Програміст  (ProgrammerModel), з 22.10.2026
      Дарина Кравченко         Програміст  (ProgrammerModel), з 22.10.2026
      Дмитро Романюк           Програміст  (ProgrammerModel), з 22.10.2026
      Марія Савчук             Програміст  (ProgrammerModel), з 22.10.2026
      Тарас Олійник            Програміст  (ProgrammerModel), з 22.10.2026
      Андрій Романюк           Художник    (ArtistModel), з 22.10.2026
      Дмитро Олійник           Художник    (ArtistModel), з 22.10.2026
    Зворотний бік зв'язку: Владислав Ткаченко працює в 6 проєктах: Sky Tactics, Shadow Odyssey, Sky Racer, Shadow Quest, Shadow Knights, Crystal Legends

--- Один-до-багатьох: Programmers → BugReports (виконавець із найбільшою кількістю багів)
    Максим Гнатюк (Senior GDScript) — багів: 5, виправлено: 0

--- Наслідування «таблиця на тип» (TPT): SQL, який генерує EF Core для запиту до Programmers
    SELECT "e"."Id", "e"."BaseSalary", "e"."FullName", "e"."HireDate", "e"."PublicId", "p"."CommitsCount", "p"."Grade", "p"."PrimaryLanguage"
    FROM "Employees" AS "e"
    INNER JOIN "Programmers" AS "p" ON "e"."Id" = "p"."Id"
    WHERE "p"."Grade" = 'Lead'
    ORDER BY "e"."FullName"

================================================================================================
6. Агрегація на стороні БД (LINQ to Entities → SQL)
================================================================================================
  Показник                                        Мін           Макс        Середнє
  ------------------------------------------------------------------------------
  Базова ставка, грн                       19 000,00      60 000,00      37 695,00
  Бюджет проєкту, грн                      60 000,00   1 993 000,00   1 052 700,00
  Коміти програміста                            37,0        4 947,0        2 535,9

Проєкти за статусом (GROUP BY у SQL):
  Cancelled   проєктів:  12, сумарний бюджет:       12 230 000 грн
  Concept     проєктів:  10, сумарний бюджет:       12 498 000 грн
  Production  проєктів:   9, сумарний бюджет:        5 799 000 грн
  Prototype   проєктів:  12, сумарний бюджет:       16 770 000 грн
  Released    проєктів:   8, сумарний бюджет:        9 015 000 грн
```

```
  Testing     проєктів:   9, сумарний бюджет:        6 850 000 грн
SQL цього запиту:
    SELECT "g"."Status", COUNT(*) AS "Count", COALESCE(SUM("g"."Budget"), 0.0) AS "Budget"
    FROM "GameProjects" AS "g"
    GROUP BY "g"."Status"
    ORDER BY "g"."Status"

Баги за критичністю: Critical — відкрито 35, виправлено 14; High — відкрито 27, виправлено 12; Low — відкрито 23,
виправлено 5; Medium — відкрито 26, виправлено 17

================================================================================================
7. Підсумок: вміст таблиць БД
================================================================================================
  Employees              200 рядків
  Programmers            102 рядків
  Artists                 98 рядків
  GameProjects            60 рядків
  StorePages              26 рядків
  ProjectAssignments     329 рядків
  BugReports             159 рядків
Розмір файлу БД: 196 608 байт
Роботу програми завершено.
```

6. Додаткове завдання: MongoDB

Команда запуску: `dotnet run --project GameStudio.Console -- nosql` (сервер MongoDB 8.0.34 на `localhost:27017`).

```
Лабораторна робота №3, додаткове завдання — MongoDB (GameStudio.Nosql)
Запуск: 22.10.2026 00:46

=============================================================================================================
1. Підключення до MongoDB
=============================================================================================================
Рядок підключення: mongodb://localhost:27017; база даних: gamestudio
Сервер MongoDB, версія: 8.0.34

=============================================================================================================
2. Перенесення даних із SQLite (7 таблиць) у MongoDB (2 колекції) через MongoRepository.AddAsync
=============================================================================================================
Колекція employees: 200 документів (Programmer і Artist в одній колекції, тип — у полі _t)
Колекція projects:  60 документів (вкладено: сторінок — 26, учасників команд — 329, багів — 159)

=============================================================================================================
3. IRepository<T> для MongoDB: GetByIdAsync / GetAllAsync / AddAsync / Update / Delete
=============================================================================================================
GetByIdAsync(2) — проєкт «Lost Tactics» з усіма вкладеними даними (один документ, без JOIN):
  {
    "_id" : 2,
    "PublicId" : UUID("1c31ba9b-c44c-4b8e-82f9-0917a195d97c"),
    "Title" : "Lost Tactics",
    "Genre" : "Action RPG",
    "Engine" : "Unity",
    "Budget" : { "$numberDecimal" : "1568000" },
    "Spent" : { "$numberDecimal" : "1545070.79" },
    "Status" : "Cancelled",
    "Team" : [{
        "EmployeeId" : 47,
        "FullName" : "Дмитро Олійник",
        "Role" : "Художник"
      }, {
        "EmployeeId" : 90,
        "FullName" : "Андрій Романюк",
        "Role" : "Художник"
      }, {
        "EmployeeId" : 116,
        "FullName" : "Дарина Кравченко",
        "Role" : "Програміст"
      }, {
        "EmployeeId" : 145,
        "FullName" : "Андрій Бондар",
        "Role" : "Програміст"
      }, {
        "EmployeeId" : 154,
        "FullName" : "Тарас Олійник",
        "Role" : "Програміст"
      }, {
        "EmployeeId" : 179,
        "FullName" : "Марія Савчук",
        "Role" : "Програміст"
      }, {
        "EmployeeId" : 182,
```

```
        "FullName" : "Дмитро Романюк",
        "Role" : "Програміст"
      }, {
        "EmployeeId" : 184,
        "FullName" : "Артем Шевчук",
        "Role" : "Програміст"
      }],
    "Bugs" : [{
        "BugId" : 2,
        "Title" : "Камера скидається після виходу",
        "Severity" : "High",
        "IsFixed" : false,
        "CreatedAt" : { "$date" : "2026-07-29T00:00:00Z" },
        "AssigneeId" : 145,
        "AssigneeName" : "Андрій Бондар"
      }, {
        "BugId" : 3,
        "Title" : "Мінікарта мерехтить у темних локаціях",
        "Severity" : "Medium",
        "IsFixed" : false,
        "CreatedAt" : { "$date" : "2026-08-22T00:00:00Z" },
        "AssigneeId" : 154,
        "AssigneeName" : "Тарас Олійник"
      }, {
        "BugId" : 4,
        "Title" : "Гравець мерехтить у темних локаціях",
        "Severity" : "Medium",
        "IsFixed" : false,
        "CreatedAt" : { "$date" : "2026-09-03T00:00:00Z" }
      }, {
        "BugId" : 5,
        "Title" : "Головне меню провалюється крізь текстури",
        "Severity" : "High",
        "IsFixed" : true,
        "CreatedAt" : { "$date" : "2026-09-06T00:00:00Z" },
        "AssigneeId" : 179,
        "AssigneeName" : "Марія Савчук"
      }]
  }

AddAsync(новий програміст з Id = 0) → отримав _id = 201 (лічильник у колекції counters)
Update → GetByIdAsync(201): Тестовий Програміст, Middle, комітів: 15
Delete → GetByIdAsync(201) → KeyNotFoundException: Документ EmployeeDocument з _id 201 не знайдено в колекції employees.
GetAllAsync: працівників — 200, проєктів — 60

=============================================================================================
4. Запити до вкладених документів і агрегація на сервері MongoDB
=============================================================================================
Проєкти з невиправленими критичними багами (фільтр по масиву Bugs): 30
  _id   1: «Frozen Knights» — критичних відкритих: 1
  _id   8: «Wild Dungeon» — критичних відкритих: 1
  _id   9: «Iron Islands» — критичних відкритих: 1

Проєкти за статусом (конвеєр агрегації $group на сервері):
  Cancelled   проєктів:  12, середній бюджет:      1 019 167 грн
  Concept     проєктів:  10, середній бюджет:      1 249 800 грн
  Production  проєктів:   9, середній бюджет:        644 333 грн
  Prototype   проєктів:  12, середній бюджет:      1 397 500 грн
  Released    проєктів:   8, середній бюджет:      1 126 875 грн
  Testing     проєктів:   9, середній бюджет:        761 111 грн
```

```
Фільтр за типом (поле _t = "Programmer"), найдавніше прийнятий: Софія Шевчук, з 18.01.2015
Роботу програми завершено.
```

Скріншоти об'єктів у MongoDB (оболонка mongosh, база `gamestudio`):

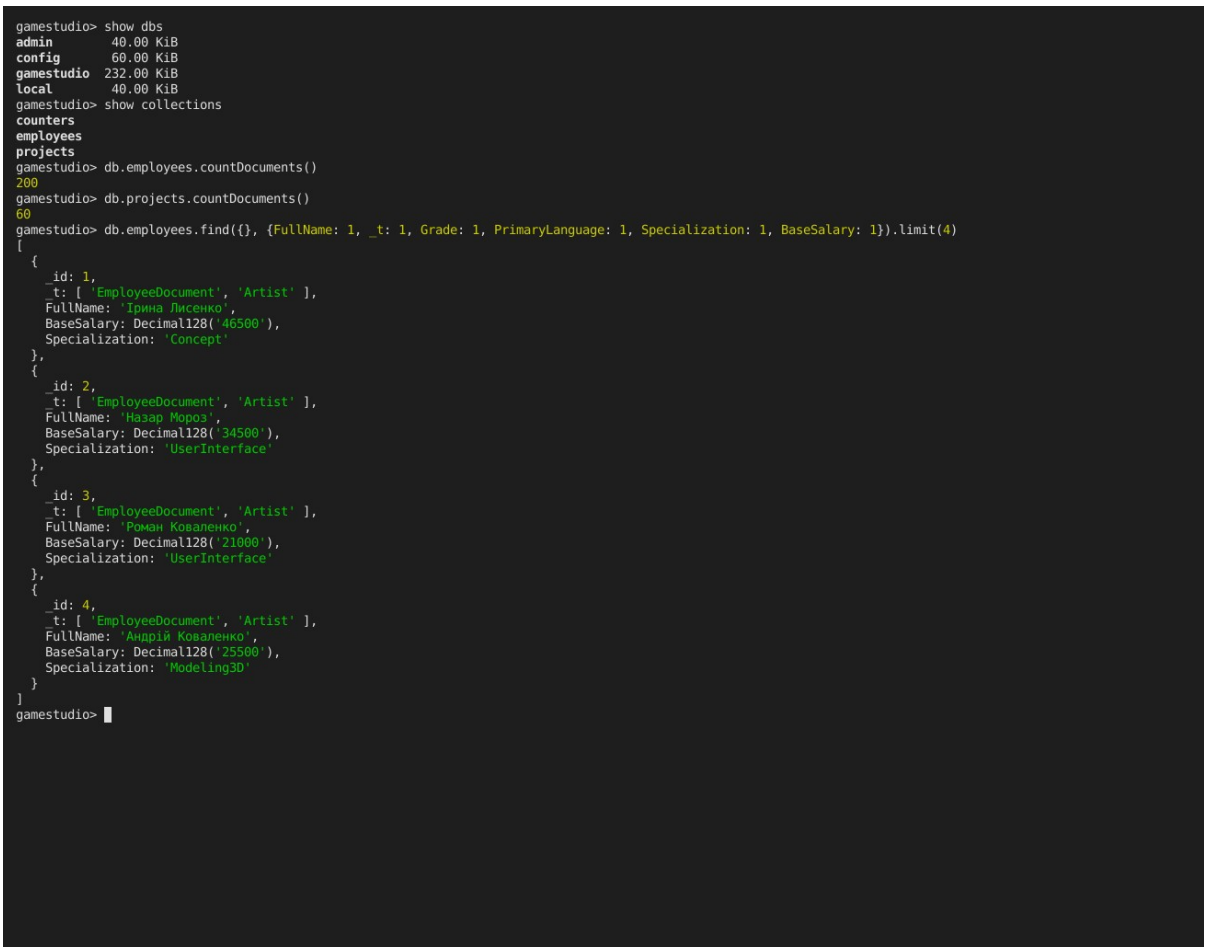


Рисунок 13 — Колекції бази gamestudio і документи працівників (поле-дискримінатор _t)

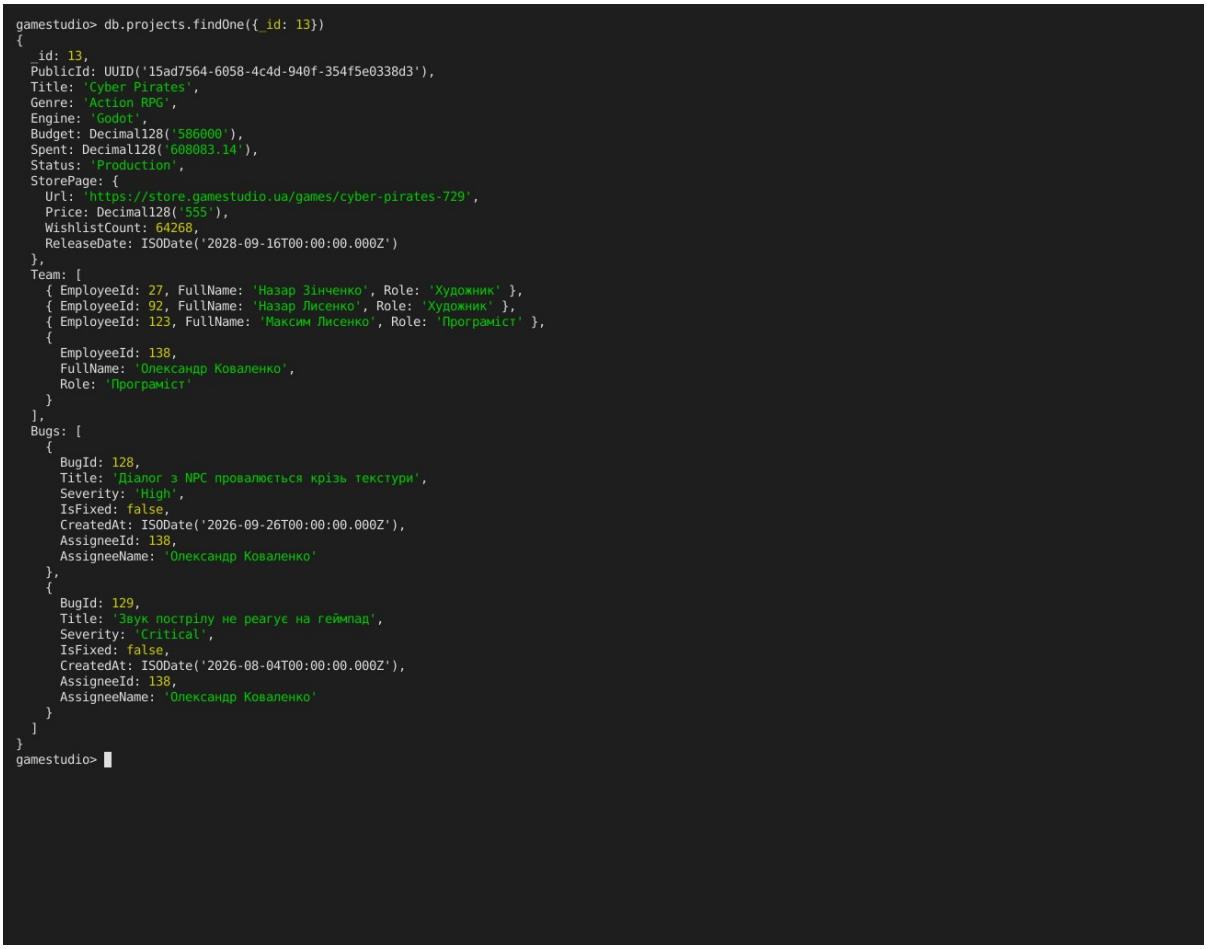


Рисунок 14 — Документ проєкту з вкладеними StorePage, Team і Bugs

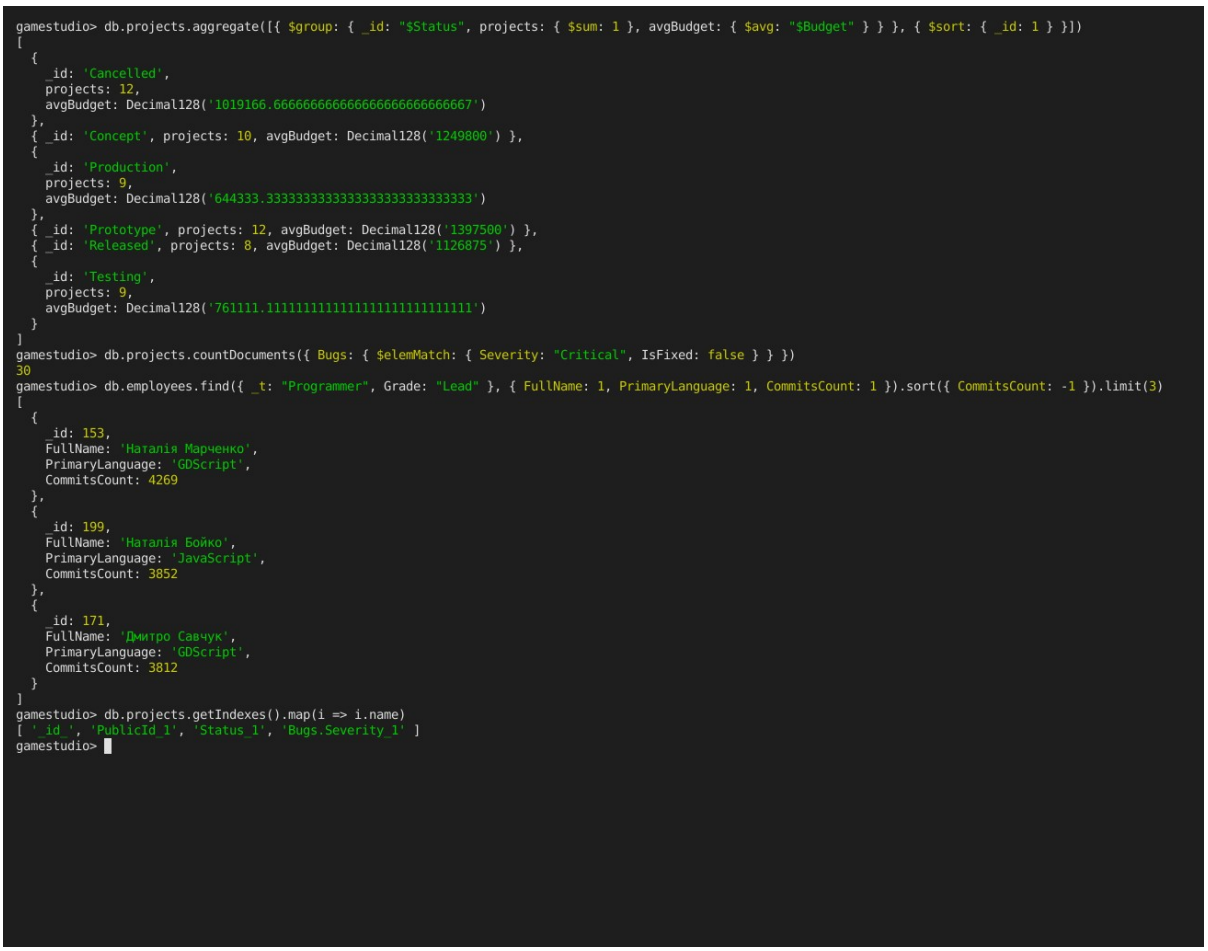


Рисунок 15 — Агрегація $group, пошук по вкладеному масиву ($elemMatch), фільтр за типом, індекси

7. Результати модульних тестів

Команда `dotnet test`: пройдено 49 з 49 тестів. Нові тести ЛР3 (CRUD-сервіс через репозиторій, SQLite):

```
✔ RepositoryCrudServiceTests.BugReport_WithUnknownProject_ThrowsInvalidOperationException
✔ RepositoryCrudServiceTests.CreateAsync_DuplicateId_ReturnsFalse
✔ RepositoryCrudServiceTests.CreateAsync_FromManyTasksConcurrently_IsSerializedSafely
✔ RepositoryCrudServiceTests.CreateAsync_ThenSaveAsync_PersistsEmployeesWithTheirTypes
✔ RepositoryCrudServiceTests.GameProject_RoundTrip_PreservesStorePageAndTeam
✔ RepositoryCrudServiceTests.ReadAllAsync_Paging_ReturnsRequestedPageInInsertionOrder
✔ RepositoryCrudServiceTests.ReadAsync_MissingId_ThrowsKeyNotFoundException
✔ RepositoryCrudServiceTests.RemoveAsync_Project_DeletesRelatedRowsByCascade
✔ RepositoryCrudServiceTests.UpdateAsync_MissingElement_ReturnsFalse
✔ RepositoryCrudServiceTests.UpdateAsync_ThenSaveAsync_PersistsChanges

Total tests: 49
     Passed: 49
```

8. Відповіді на контрольні запитання

1. Що таке реляційні бази даних? Що таке СУБД? Які СУБД ви знаєте?

Реляційна база даних зберігає дані у вигляді відношень — таблиць, що складаються з рядків (записів) і стовпців (атрибутів) з визначеними типами. Модель запропонував Е. Кодд (1970). Кожен рядок ідентифікується первинним ключем, таблиці пов'язуються зовнішніми ключами. Дані читаються і змінюються мовою SQL, а цілісність забезпечують обмеження (NOT NULL, UNIQUE, CHECK, FOREIGN KEY) і транзакції з властивостями ACID.

СУБД (система керування базами даних) — програмне забезпечення, яке створює БД і керує нею. Вона зберігає дані, виконує запити, контролює цілісність і права доступу, керує транзакціями та одночасним доступом багатьох користувачів, робить резервні копії.

Реляційні СУБД: Microsoft SQL Server, PostgreSQL, MySQL / MariaDB, Oracle Database, IBM Db2, SQLite (вбудована: уся БД — один файл, окремий сервер не потрібен). **Нереляційні:** MongoDB, Redis, Cassandra, Neo4j. У лабораторній використано SQLite (файл `Database/gamestudio.db` у репозиторії) і MongoDB у додатковому завданні.

2. Що позначає термін таблиця у реляційній БД? Які існують зв'язки у реляційній БД?

Таблиця (відношення) — набір рядків однакової структури. Стовпці мають ім'я і тип даних, кожен рядок описує один об'єкт і однозначно визначається первинним ключем, порядок рядків не визначений. Наприклад, таблиця `GameProjects` має стовпці Id, PublicId, Title, Genre, Engine, Budget, Spent, Status, а кожен рядок — це один проєкт.

Зв'язок	Суть	Приклад у БД
один-до-одного (1:1)	запису однієї таблиці відповідає не більше одного запису іншої	GameProjects ↔ StorePages (унікальний FK GameProjectId)
один-до-багатьох (1:N)	одному запису відповідає багато записів іншої таблиці	GameProjects → BugReports; Programmers → BugReports
багато-до-багатьох (N:M)	реалізується проміжною таблицею з двома FK	GameProjects ↔ Employees через ProjectAssignments
самопосилання	FK посилається на ту саму таблицю	напр. Employees.ManagerId → Employees.Id

3. Що таке ERD-діаграма? Як її створити та для чого вона потрібна?

ERD (Entity-Relationship Diagram, діаграма «сутність — зв'язок») — графічна модель структури БД. На ній показано сутності (таблиці) з атрибутами (стовпці, типи, ключі) і зв'язки між ними з кардинальністю. Поширені нотації: Чена, «воронячої лапки» (Crow's Foot), UML.

Як створити: вручну в редакторах діаграм (draw.io, dbdiagram.io, Lucidchart) або в інструментах СУБД (MySQL Workbench, SSMS, DBeaver, DataGrip). Також діаграму можна згенерувати з наявної БД (зворотне проєктування). У лабораторній ERD (рисунок 1) побудовано за фактичною схемою SQLite: таблиці, стовпці та зовнішні ключі прочитано з метаданих (`PRAGMA table_info`, `PRAGMA foreign_key_list`) і намальовано в нотації Crow's Foot за допомогою Graphviz.

Для чого: щоб спроєктувати БД до написання коду, задокументувати її структуру, обговорити модель у команді, перевірити нормалізацію і правильність зв'язків.

4. Що таке DbContext і яку роль він відіграє в роботі з базою даних?

`DbContext` — центральний клас Entity Framework Core, що представляє сеанс роботи з БД. Він:

- містить набори сутностей `DbSet<T>` — через них пишуться LINQ-запити (`context.GameProjects.Where(...)`);
- описує модель і схему БД: `OnModelCreating` + Fluent API (у нас — `ApplyConfigurationsFromAssembly` з класами папки `Configurations`);
- отримує налаштування підключення через `DbContextOptions` (провайдер і рядок підключення: `UseSqlite("Data Source=...")`);
- перетворює LINQ на SQL і матеріалізує результати в об'єкти;
- відстежує зміни завантажених сутностей (Change Tracker) і в `SaveChanges` / `SaveChangesAsync` записує їх в одній транзакції — це шаблон «одиниця роботи»;
- керує схемою: `Database.MigrateAsync()`, `EnsureDeletedAsync()` тощо.

DbContext має бути «короткоживучим» і не є потокобезпечним. Тому в консольному застосунку кожен сервіс отримує свій `GameStudioContext`, а `CrudServiceAsync<TDomain, TModel>` впорядковує звернення до нього семафором.

5. Що таке зв'язки один-до-одного, один-до-багатьох і багато-до-багатьох у контексті Entity Framework Core?

- **Один-до-одного:** обидві сутності мають навігаційну властивість-посилання (`GameProjectModel.StorePage` і `StorePageModel.GameProject`). Зовнішній ключ розташований у залежній сутності (`StorePageModel.GameProjectId`), і EF створює для нього унікальний індекс.
- **Один-до-багатьох:** головна сутність має колекцію (`GameProjectModel.Bugs`), залежна — посилання і зовнішній ключ (`BugReportModel.GameProject`, `GameProjectId`). Зв'язок буває обов'язковим (FK типу int) або необов'язковим (FK типу int?, як `AssigneeId`).
- **Багато-до-багатьох:** обидві сутності мають колекції (`GameProjectModel.TeamMembers` і `EmployeeModel.Projects`), а в БД зв'язок зберігається в проміжній таблиці. Починаючи з EF Core 5, її можна не описувати класом, але якщо потрібні додаткові поля, створюють сутність-зв'язок (`ProjectAssignmentModel` з Role і AssignedAt).

Для кожного зв'язку EF Core налаштовує поведінку при видаленні (`DeleteBehavior.Cascade`, `SetNull`, `Restrict`). Пов'язані дані завантажуються через `Include` / `ThenInclude` (жадібне завантаження), явне або ліниве завантаження, або автоматично через `AutoInclude`.

6. Як реалізувати зв'язок один-до-одного за допомогою Fluent API?

Методи `HasOne` / `WithOne` задають навігаційні властивості з обох боків. У `HasForeignKey<T>` обов'язково треба вказати тип залежної сутності, бо для 1:1 EF не може сам визначити, на чиєму боці зовнішній ключ:

```
public class StorePageModel
{
    public int Id { get; set; }
    public int GameProjectId { get; set; }                 // FK
    public GameProjectModel GameProject { get; set; } = null!;
    ...
}

// GameProjectModelConfiguration.Configure(EntityTypeBuilder<GameProjectModel> builder)
builder.HasOne(p => p.StorePage)
    .WithOne(s => s.GameProject)
    .HasForeignKey<StorePageModel>(s => s.GameProjectId)
    .OnDelete(DeleteBehavior.Cascade);
```

Міграція створює стовпець `StorePages.GameProjectId` із зовнішнім ключем і унікальний індекс `IX_StorePages_GameProjectId`. Саме він гарантує, що в проєкту не більше однієї сторінки.

7. Наведіть приклад реалізації зв'язку багато-до-багатьох у EF Core.

Найпростіший варіант (EF Core 5+): колекції з обох боків, проміжну таблицю EF створить автоматично:

```
public class Student { public int Id { get; set; } public List<Course> Courses { get; set; }
= new(); }
public class Course  { public int Id { get; set; } public List<Student> Students { get; set;
} = new(); }

modelBuilder.Entity<Student>().HasMany(s => s.Courses).WithMany(c => c.Students);
```

У лабораторній проміжна таблиця має додаткові поля, тому використано явну сутність-зв'язок і `UsingEntity`:

```
builder.HasMany(p => p.TeamMembers)
    .WithMany(e => e.Projects)
    .UsingEntity<ProjectAssignmentModel>(
        right => right.HasOne(a => a.Employee).WithMany(e => e.Assignments)
            .HasForeignKey(a => a.EmployeeId).OnDelete(DeleteBehavior.Cascade),
        left => left.HasOne(a => a.GameProject).WithMany(p => p.Assignments)
            .HasForeignKey(a => a.GameProjectId).OnDelete(DeleteBehavior.Cascade),
        join =>
        {
            join.ToTable("ProjectAssignments");
            join.HasKey(a => new { a.GameProjectId, a.EmployeeId });   // складений PK
            join.Property(a => a.Role).IsRequired().HasMaxLength(50);
        });

// запит: команда проєкту
var project = await context.GameProjects.Include(p => p.TeamMembers).FirstAsync(p => p.Id ==
id);
```

8. У чому різниця між анотаціями (Data Annotations) та Fluent API в конфігурації моделей?

Data Annotations	Fluent API
атрибути над класами й властивостями: `[Key]`, `[Required]`, `[MaxLength(100)]`, `[Table("...")]`, `[ForeignKey]`, `[NotMapped]`	виклики методів у `OnModelCreating` або в класах `IEntityTypeConfiguration<T>`: `HasKey`, `IsRequired`, `HasMaxLength`, `ToTable`, `HasOne/WithMany`
налаштування поруч із властивістю, просто й наочно	уся конфігурація зібрана в одному місці, моделі лишаються «чистими» класами без залежності від EF
можливості обмежені	доступні всі можливості: складені й альтернативні ключі, стратегія наслідування, зв'язки з `UsingEntity`, перетворення значень, `AutoInclude`, CHECK-обмеження, поведінка видалення
нижчий пріоритет	вищий пріоритет: Fluent API > анотації > угоди (conventions)

За рекомендацією завдання в лабораторній використано лише Fluent API: класи `EmployeeModelConfiguration`, `GameProjectModelConfiguration` та інші в папці `Configurations`.

9. Як створюється та застосовується міграція в EF Core?

Потрібні пакет `Microsoft.EntityFrameworkCore.Design` і інструмент `dotnet-ef` (у нас — локальний, `.config/dotnet-tools.json`). Інструмент створює контекст через `IDesignTimeDbContextFactory` (`GameStudioContextFactory`).

```
dotnet tool restore                                          # відновити локальний dotnet-ef
dotnet ef migrations add InitialCreate --project GameStudio.Infrastructure
dotnet ef database update --project GameStudio.Infrastructure
dotnet ef migrations list --project GameStudio.Infrastructure
dotnet ef migrations script --project GameStudio.Infrastructure   # SQL-скрипт міграцій
dotnet ef database update <ПопередняМіграція>                     # відкат
dotnet ef migrations remove                                  # видалити останню
незастосовану
```

- `migrations add` порівнює поточну модель зі знімком `ModelSnapshot` і генерує клас міграції з методами `Up` (CreateTable, AddColumn, CreateIndex…) і `Down` (відкат), а також оновлює знімок.
- `database update` виконує незастосовані міграції і записує їх у таблицю `__EFMigrationsHistory`.
- Міграції можна застосовувати і з коду: `await context.Database.MigrateAsync()` (так робить консольний застосунок під час запуску).

10. Яка мета проєкту GameStudio.Infrastructure? Яку роль він відіграє у загальній структурі застосунку?

`GameStudio.Infrastructure` — інфраструктурний шар. Тут зібрано все, що стосується технології зберігання даних: моделі БД, `GameStudioContext`, конфігурації Fluent API,

міграції, реалізацію репозиторію (`Repository<T>`) і перетворювачі між доменною моделлю та моделлю БД.

Проєкт відокремлює бізнес-логіку від технічних деталей. `GameStudio.Common` (доменні класи, інтерфейси `IRepository<T>`, `ICrudServiceAsync<T>`, CRUD-сервіс) не залежить ні від EF Core, ні від SQLite. Залежності спрямовані до домену: Console → Infrastructure → Common, Nosql → Common. Тому сховище можна замінити — наприклад, SQLite на PostgreSQL або MongoDB (`GameStudio.Nosql`) — не змінюючи доменну модель і код, що працює через інтерфейси. Такий поділ відповідає багатошаровій («цибулевій», Clean) архітектурі.

11. У чому різниця між доменною моделлю (з попередньої лабораторної) та моделлю бази даних?

Доменна модель (Common/Models)	Модель БД (Infrastructure/Models)
описує предметну область і поведінку: методи `Promote()`, `Spend()`, `ChangeStatus()`, події, перевірки в конструкторах	описує структуру зберігання: прості властивості get/set без логіки
інкапсуляція: `Status`, `Spent` змінюються лише методами	усі властивості відкриті, щоб EF міг їх заповнювати
ідентифікатор — `Guid Id`; зв'язки через ідентифікатори (`ProjectId`, `TeamMemberIds`)	первинний ключ `int Id` + альтернативний `PublicId`; зовнішні ключі та навігаційні властивості для EF
не знає про таблиці й СУБД	містить елементи, яких немає в домені: проміжна сутність `ProjectAssignmentModel` з полями Role і AssignedAt

Перетворення виконують класи `EmployeeMapper`, `GameProjectMapper`, `BugReportMapper` (інтерфейс `IModelMapper<TDomain, TModel>`). Завдяки цьому схему БД можна змінювати, не ламаючи бізнес-логіку, і навпаки.

12. Яку проблему вирішує шаблон проектування Репозиторій?

Репозиторій відокремлює бізнес-логіку від способу доступу до даних. Він надає інтерфейс, схожий на колекцію об'єктів (`GetByIdAsync`, `GetAllAsync`, `AddAsync`, `Update`, `Delete`), і приховує, як саме дані зберігаються: SQL, EF Core чи MongoDB. Наслідки:

- код доступу до даних не дублюється — один узагальнений `Repository<T>` обслуговує всі моделі;
- сховище можна замінити: у лабораторній той самий `IRepository<T>` має дві реалізації — `Repository<T>` (EF Core + SQLite) і `MongoRepository<T>` (MongoDB);
- бізнес-логіку (`CrudServiceAsync<TDomain, TModel>`) легко тестувати, підставивши іншу реалізацію репозиторію;
- запити зосереджено в одному місці, їх простіше змінювати й оптимізувати (наприклад, пагінація на рівні БД).

13. У чому принципова різниця між реляційними та нереляційними базами даних?

	Реляційні (SQLite, PostgreSQL…)	**Нереляційні (MongoDB…)**
Модель даних	таблиці з рядками і стовпцями	документи (JSON/BSON), ключ-значення, стовпцеві сімейства, графи
Схема	жорстка, змінюється міграціями	гнучка: документи однієї колекції можуть мати різні поля
Зв'язки	зовнішні ключі + JOIN, нормалізація	вкладення (денормалізація) або посилання за ідентифікатором
Мова запитів	SQL	API конкретної БД (запити й конвеєри агрегації MongoDB)
Транзакції	ACID для будь-якої кількості таблиць	атомарність на рівні документа; багатодокументні транзакції — додатково
Масштабування	переважно вертикальне	горизонтальне (шардинг, репліки)

У лабораторній ті самі дані займають 7 таблиць у SQLite і лише 2 колекції в MongoDB. Документ проєкту містить сторінку в магазині, команду і баги, тому читається одним запитом без JOIN, але дані учасників команди дублюються (денормалізація). Наслідування в MongoDB реалізоване полем-дискримінатором `_t` в одній колекції, а не окремими таблицями, як у TPT.